

SBA301 — SLOT 6

REACT HOOK PRODUCT MANAGER

Hướng dẫn dự án ReactJS tổng hợp mức độ trung bình với IntelliJ IDEA

useState • useEffect • useContext • useRef • Custom Hook • Controlled Form • Immutable Update

Thông tin	Nội dung
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Lớp	SE1910
Slot	Slot 6 – Chapter 05: React Hook + Lab 01
Mức độ	Trung bình
Thời lượng dự kiến	3–5 giờ thực hành cá nhân hoặc nhóm nhỏ
IDE	IntelliJ IDEA; có thể dùng Terminal tích hợp trong IDE
Sản phẩm	Source code, README.md, screenshots, Git commit/link nếu được yêu cầu

Mục tiêu: Sinh viên xây được một ứng dụng quản lý sản phẩm hoàn chỉnh ở mức frontend, thực hành đúng React Hooks, kiểm tra từng bước và có khả năng giải thích data flow mà không phụ thuộc vào AI.

MỤC LỤC NỘI DUNG

0. THÔNG TIN DỰ ÁN

1. TÓM TẮT DỰ ÁN VÀ MỤC TIÊU

2. CHỨC NĂNG CỦA ỨNG DỤNG

3. CÔNG NGHỆ SỬ DỤNG

4. CẤU TRÚC THƯ MỤC DỰ ÁN

5. YÊU CẦU CÀI ĐẶT VÀ CHUẨN BỊ MÔI TRƯỜNG

6. TÓM TẮT DANH SÁCH CÁC BƯỚC THỰC HIỆN

7. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

8. ĐÁNH GIÁ KẾT QUẢ HOÀN THÀNH TỪNG BƯỚC

9. CHẠY VÀ KIỂM TRA ỨNG DỤNG HOÀN CHỈNH

10. LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

11. PHẦN MỞ RỘNG VỚI AI ASSISTANT

12. CHECKLIST Củng Cố Kiến Thức SLOT 6

13. RUBRIC ĐÁNH GIÁ

PHỤ LỤC A. SOURCE CODE HOÀN CHỈNH

PHỤ LỤC B. README MẪU & EVIDENCE PACKAGE

PHỤ LỤC C. QUICK REFERENCE

0. THÔNG TIN DỰ ÁN

Mục	Thông tin
Tên dự án	React Hook Product Manager
Slot liên quan	Slot 6 – React Hook
Mức độ	Trung bình
Thời lượng	3–5 giờ
Hình thức	Cá nhân hoặc nhóm 2–3 sinh viên
Frontend	ReactJS + JavaScript/JSX + React-Bootstrap
Persistence	localStorage; không có backend thật
IDE	IntelliJ IDEA
Mục đích	Củng cố React Hook và Lab 01
Sản phẩm nộp	Source, README, screenshot, Git commit/link, video nếu yêu cầu

Phạm vi: Dự án tập trung vào frontend và React Hook. Không dùng Router, Axios, Spring Boot, REST API hay database trong phần core. Các công nghệ đó thuộc các slot tiếp theo.

1. TÓM TẮT DỰ ÁN VÀ MỤC TIÊU

React Hook Product Manager là ứng dụng quản lý danh sách sản phẩm chạy hoàn toàn trên trình duyệt. Người dùng có thể xem, thêm, sửa, xóa, tìm kiếm, lọc theo danh mục, xem thống kê, đổi theme và giữ dữ liệu sau khi tải lại trang bằng localStorage.

1.1 Mục tiêu kiến thức

Hiểu và sử dụng useState cho dữ liệu thay đổi và ảnh hưởng UI; useEffect cho side effect/synchronization; useContext cho theme dùng chung; useRef cho DOM reference; Custom Hook để tái sử dụng logic; controlled form, validation và immutable update.

Phân biệt state thật sự với dữ liệu có thể suy ra từ state/props. Trong project này, danh sách đã lọc và thống kê được tính từ products/search/category thay vì lưu thành một state khác.

1.2 Mục tiêu kỹ năng

Tạo và chạy Vite project trong IntelliJ; tổ chức components/context/hooks/data/styles; kiểm tra từng milestone; đọc lỗi terminal và browser console; dùng Git checkpoint; giải thích data flow và Hook ownership.

1.3 Kết quả đầu ra

Ứng dụng chạy được, có CRUD frontend, tìm kiếm/lọc/thống kê, theme, localStorage, focus input, document title effect, cleanup timer, custom hook, README và test evidence.

2. CHỨC NĂNG CỦA ỨNG DỤNG

Nhóm chức năng	Mô tả	Hook/Kỹ thuật liên quan
Hiển thị danh sách	Tên, giá, danh mục, số lượng, giá trị tiền	useState/useLocalStorage
Thêm sản phẩm	Controlled form + validation	useState
Sửa sản phẩm	Nạp dữ liệu vào form và cập nhật bằng map	useState + useEffect

Nhóm chức năng	Mô tả	Hook/Kỹ thuật liên quan
Xóa sản phẩm	Xác nhận và cập nhật bằng filter	useState
Tìm kiếm	Lọc theo tên; focus tự động	useState + useRef
Lọc danh mục	Dropdown category	useState
Thống kê	Số mặt hàng, tổng số lượng, tổng giá trị	Derived data
Dark/Light	Theme dùng chung toàn app	useContext + useState
Persistence	Reload vẫn giữ products/theme	Custom Hook + useEffect
Document title	Title phản ánh số sản phẩm	useEffect
Feedback	Thông báo tự ẩn sau 2.5s	useEffect + cleanup
Empty state	Thông báo khi không có record phù hợp	Conditional rendering

3. CÔNG NGHỆ SỬ DỤNG

Công nghệ	Vai trò
ReactJS	Xây dựng UI bằng function component và Hooks
JavaScript/JSX	Ngôn ngữ và biểu diễn UI
Vite	Scaffold, dev server, build
Node.js + npm	Runtime/tooling và package management
IntelliJ IDEA	IDE, Terminal, npm Run Configuration, code inspection
React-Bootstrap + Bootstrap	UI components và responsive layout
localStorage	Persistence phía trình duyệt
Git	Checkpoint và evidence
Browser DevTools	Console, Application/Storage, kiểm tra UI

Lưu ý tương thích 2026: Prompt môn học dùng Node 21. Tuy nhiên Vite hiện hành yêu cầu Node 20.19+ hoặc 22.12+; Node 21 không nằm trong dải được Vite hiện hành công bố. Nếu lớp buộc dùng Node 21, hãy dùng Vite 5 theo baseline đã thống nhất; nếu được phép cập nhật môi trường, ưu tiên Node 22 LTS và Vite hiện hành. Không tự nâng phiên bản giữa buổi nếu chưa có hướng dẫn của giảng viên.

IntelliJ IDEA: IntelliJ IDEA hiện có tích hợp React/Vite khi các plugin JavaScript and TypeScript, React và Vite được bật. Nếu máy/lab thiếu plugin hoặc tính năng UI, vẫn có thể mở project và dùng Terminal tích hợp (Alt+F12) để chạy npm scripts.

4. CẤU TRÚC THƯ MỤC DỰ ÁN

```

react-hook-product-manager/
├── public/
├── src/
│   ├── components/
│   │   ├── Header.jsx
│   │   ├── ProductForm.jsx
│   │   ├── ProductList.jsx
│   │   ├── ProductItem.jsx
│   │   └── SearchBox.jsx

```

```

├── CategoryFilter.jsx
├── ProductStats.jsx
├── ThemeToggle.jsx
├── context/
│   └── ThemeContext.jsx
├── hooks/
│   └── useLocalStorage.js
├── data/
│   └── initialProducts.js
├── styles/
│   └── app.css
├── App.jsx
├── main.jsx
├── .gitignore
├── package.json
├── README.md
└── vite.config.js

```

Thư mục/File	Trách nhiệm
components/	UI nhỏ, có responsibility rõ ràng; nhận props và phát event qua callback
context/	Shared theme state và custom useTheme hook
hooks/	Logic Hook tái sử dụng; ở đây là persistence localStorage
data/	Mock data ban đầu, không phải database
styles/	CSS app-level
App.jsx	State ownership chính, CRUD orchestration, search/filter, effects
main.jsx	React entry point và Provider composition

5. YÊU CẦU CÀI ĐẶT VÀ CHUẨN BỊ MÔI TRƯỜNG

5.1 Kiểm tra công cụ

Mở Terminal trong IntelliJ bằng Alt+F12 hoặc View > Tool Windows > Terminal. Chạy các lệnh sau và ghi lại kết quả.

```

node -v
npm -v
npx --version
git --version

```

PASS: Node/npm trả về version, Git hoạt động, terminal đang ở đúng thư mục làm việc.

5.2 Tạo project

Chọn một trong hai đường đi tùy môi trường lớp.

```

# A. Môi trường hiện hành (khuyến nghị nếu dùng Node 22.12+)
npm create vite@latest react-hook-product-manager -- --template react

# B. Nếu lớp buộc giữ Node 21, pin Vite 5
npm create vite@5 react-hook-product-manager -- --template react

cd react-hook-product-manager
npm install

```

5.3 Cài UI library

Cài React-Bootstrap và Bootstrap, sau đó import CSS Bootstrap ở main.jsx.

```
npm install react-bootstrap bootstrap
```

5.4 IntelliJ Run Configuration

Run > Edit Configurations > + > npm; package.json trở tới project; Command = run; Scripts = dev. Nếu giao diện khác phiên bản, dùng Terminal và npm run dev là đủ.

Baseline Gate: Không sửa source trước khi project mặc định chạy bằng npm run dev và browser mở được URL Vite local.

6. TÓM TẮT DANH SÁCH CÁC BƯỚC THỰC HIỆN

Bước	Milestone	Phiên bản
0	Baseline môi trường và project	V0
1	Dọn code mặc định, import Bootstrap/CSS	V0
2	Tạo cấu trúc thư mục	V0
3	Tạo dữ liệu initialProducts	V1
4	Tạo custom hook useLocalStorage	V1
5	Tạo ThemeContext và useTheme	V1
6	Tạo Header + ThemeToggle	V2
7	Tạo ProductItem	V2
8	Tạo ProductList	V2
9	Đưa products vào App bằng useLocalStorage	V3
10	Tạo ProductForm controlled + validation	V3
11	Thêm sản phẩm	V3
12	Xóa sản phẩm	V4
13	Sửa sản phẩm	V4
14	Tạo SearchBox + useRef focus	V4
15	Tạo CategoryFilter	V5
16	Tạo derived filtering	V5
17	Tạo ProductStats	V5
18	Thêm useEffect cập nhật document.title	V6
19	Thêm feedback effect + cleanup	V6
20	Hoàn thiện CSS/theme/responsive	V6
21	Kiểm tra Rules of Hooks + localStorage	V7
22	Git checkpoint + build	V7
23	Final Verification Gate	V7

7. HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

Quy tắc thực hành: Mỗi bước theo chu trình ACTION → RUN → VERIFY → nếu FAIL thì sửa lỗi đầu tiên có ý nghĩa → chỉ chuyển bước khi PASS. Không dán toàn bộ project rồi chạy một lần cuối.

Bước 0. Baseline môi trường và project

Mục tiêu: Xác nhận đúng thư mục, dependency và dev server trước khi viết code.

Thao tác

- Mở project trong IntelliJ.
- Mở Terminal và chạy npm install nếu node_modules chưa có.
- Chạy npm run dev.
- Mở URL Vite được terminal in ra.

Kết quả mong đợi: Ứng dụng Vite mặc định hiển thị, terminal không có lỗi blocking.

Cách kiểm tra: node -v, npm -v, npm run dev đều PASS; browser load được app.

Lỗi thường gặp: Nếu npm báo thiếu package.json, kiểm tra lại working directory.

Trạng thái project: V0 – Baseline project chạy được

```
npm install
npm run dev
```

Bước 1. Dọn code mặc định và import Bootstrap

Mục tiêu: Chuẩn bị entry point sạch và bảo đảm Bootstrap CSS được nạp.

Thao tác

- Xóa CSS/demo assets không dùng.
- Tạo src/styles/app.css.
- Giữ App.jsx tạm thời tối giản.
- Import bootstrap CSS ở main.jsx.

Kết quả mong đợi: App vẫn render sau khi dọn source.

Cách kiểm tra: Browser không blank; Console không có import error.

Lỗi thường gặp: Sai đường dẫn CSS hoặc xóa nhầm main.jsx/index.html.

Trạng thái project: V1 – Project sạch và có Bootstrap

main.jsx (phiên bản cuối có thể dùng ngay từ bước này)

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "./styles/app.css";
import App from "./App";
import { ThemeProvider } from "../context/ThemeContext";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </StrictMode>
);
```

src/styles/app.css

```
html, body, #root { min-height: 100%; }
body { margin: 0; }
.app-shell { background: #f6f7f9; transition: background-color .2s ease; }
.app-shell[data-bs-theme="dark"] { background: #17191c; }
.card { border-radius: 0.8rem; }
.navbar-brand { font-weight: 700; }
.form-control, .form-select, .btn { border-radius: 0.6rem; }
@media (max-width: 576px) {
  .navbar-brand { font-size: 1rem; }
}
```

Bước 2. Tạo cấu trúc thư mục

Mục tiêu: Tách concerns để component, context, hooks và data có vị trí rõ ràng.

Thao tác

- Trong src tạo components, context, hooks, data, styles.
- Không đặt tất cả logic vào App.jsx.

Kết quả mong đợi: Cấu trúc giống sơ đồ ở Phần 4.

Cách kiểm tra: Project tool window hiển thị đúng folder/file.

Lỗi thường gặp: Tên file khác chữ hoa/thường có thể gây lỗi import trên hệ thống khác.

Trạng thái project: V1 – Architecture skeleton

Bước 3. Tạo dữ liệu mẫu initialProducts

Mục tiêu: Có nguồn mock data để UI hoạt động trước backend.

Thao tác

- Tạo src/data/initialProducts.js.
- Copy code đầy đủ bên dưới.
- Không coi file này là database.

Kết quả mong đợi: Có 5 sản phẩm với id ổn định và dữ liệu hợp lệ.

Cách kiểm tra: Import thử ở App sau bước 9 hoặc kiểm tra syntax.

Lỗi thường gặp: Sai export/import name; trùng id.

Trạng thái project: V1 – Static data ready

src/data/initialProducts.js

```
export const initialProducts = [
  { id: 1, name: "Mechanical Keyboard", category: "Accessories", price: 1490000, quantity: 8 },
  { id: 2, name: "Wireless Mouse", category: "Accessories", price: 690000, quantity: 15 },
  { id: 3, name: "27-inch Monitor", category: "Display", price: 5290000, quantity: 4 },
  { id: 4, name: "USB-C Hub", category: "Connectivity", price: 890000, quantity: 11 },
  { id: 5, name: "Laptop Stand", category: "Workspace", price: 790000, quantity: 9 }
];
```

Bước 4. Tạo custom hook useLocalStorage

Mục tiêu: Tái sử dụng logic persistence cho products và theme.

Thao tác

- Tạo src/hooks/useLocalStorage.js.
- Khởi tạo state bằng lazy function.
- Đọc JSON an toàn bằng try/catch.
- Đồng bộ localStorage trong useEffect với [key, value].

Kết quả mong đợi: Hook trả về [value, setValue] như useState nhưng tự lưu browser storage.

Cách kiểm tra: Sau khi tích hợp, DevTools > Application > Local Storage có key tương ứng.

Lỗi thường gặp: Quên JSON.parse/stringify; key sai; localStorage dữ liệu cũ không hợp lệ.

Trạng thái project: V2 – Custom Hook ready

src/hooks/useLocalStorage.js

```
import { useEffect, useState } from "react";

export default function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const storedValue = localStorage.getItem(key);
      return storedValue !== null ? JSON.parse(storedValue) : initialValue;
    } catch (error) {
      console.error(`Cannot read localStorage key: ${key}`, error);
      return initialValue;
    }
  });

  useEffect(() => {
    try {
      localStorage.setItem(key, JSON.stringify(value));
    } catch (error) {
      console.error(`Cannot write localStorage key: ${key}`, error);
    }
  }, [key, value]);
}
```

```
return [value, setValue];
}
```

Hook reasoning: useLocalStorage không phải database. Nó chỉ lưu state trong browser của người dùng. Khi chuyển sang Spring Boot, persistence sẽ qua HTTP/REST thay vì localStorage.

Bước 5. Tạo ThemeContext và custom useTheme

Mục tiêu: Thực hành createContext, Provider, useContext và state dùng chung có chủ đích.

Thao tác

- Tạo src/context/ThemeContext.jsx.
- Provider sở hữu theme.
- toggleTheme dùng functional update.
- useTheme kiểm tra component có nằm dưới Provider không.

Kết quả mong đợi: Các descendant trong ThemeProvider có thể đọc theme và gọi toggleTheme.

Cách kiểm tra: Nếu render ThemeToggle ngoài Provider phải xuất hiện lỗi giải thích rõ.

Lỗi thường gặp: Quên bọc ThemeProvider; import sai path.

Trạng thái project: V3 – Context ready

src/context/ThemeContext.jsx

```
import { createContext, useContext } from "react";
import useLocalStorage from "../hooks/useLocalStorage";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useLocalStorage("sba301-theme", "light");

  const toggleTheme = () => {
    setTheme((current) => (current === "light" ? "dark" : "light"));
  };

  const value = { theme, toggleTheme };

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme must be used inside ThemeProvider");
  }
  return context;
}
```

Bước 6. Tạo Header và ThemeToggle

Mục tiêu: Dùng Context trong UI thật.

Thao tác

- Tạo ThemeToggle.jsx và Header.jsx.
- ThemeToggle đọc context bằng useTheme.
- Header chỉ chịu trách nhiệm header UI.

Kết quả mong đợi: Navbar hiển thị brand và nút đổi theme.

Cách kiểm tra: Click button đổi label Dark mode/Light mode.

Lỗi thường gặp: Theme đổi value nhưng UI chưa đổi nếu App chưa gắn data-bs-theme; hoàn tất ở bước 20.

Trạng thái project: V3 – Shared theme interaction

src/components/ThemeToggle.jsx

```
import Button from "react-bootstrap/Button";
import { useTheme } from "../context/ThemeContext";

export default function ThemeToggle() {
  const { theme, toggleTheme } = useTheme();

  return (
    <Button variant="outline-light" size="sm" onClick={toggleTheme}>
      {theme === "light" ? "Dark mode" : "Light mode"}
    </Button>
  );
}
```

src/components/Header.jsx

```
import Container from "react-bootstrap/Container";
import Navbar from "react-bootstrap/Navbar";
import ThemeToggle from "../ThemeToggle";

export default function Header() {
  return (
    <Navbar bg="primary" data-bs-theme="dark" className="shadow-sm">
      <Container>
        <Navbar.Brand>React Hook Product Manager</Navbar.Brand>
        <ThemeToggle />
      </Container>
    </Navbar>
  );
}
```

Bước 7. Tạo ProductItem

Mục tiêu: Thiết kế reusable component cho một record.

Thao tác

- Tạo ProductItem.jsx.
- Nhận product, onEdit, onDelete qua props.
- Không sửa prop bên trong child.

Kết quả mong đợi: Một card có name/category/price/quantity và Edit/Delete.

Cách kiểm tra: Render tạm một ProductItem hoặc kiểm tra sau bước ProductList.

Lỗi thường gặp: Nhầm callback onEdit(product) thành onEdit; hoặc gọi onDelete ngay khi render.

Trạng thái project: V3 – Reusable item component

src/components/ProductItem.jsx

```
import Button from "react-bootstrap/Button";
import Card from "react-bootstrap/Card";

const money = new Intl.NumberFormat("vi-VN", {
  style: "currency",
  currency: "VND"
});

export default function ProductItem({ product, onEdit, onDelete }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Body className="d-flex flex-column">
        <Card.Title>{product.name}</Card.Title>
        <Card.Subtitle className="mb-2 text-secondary">
          {product.category}
        </Card.Subtitle>
        <div className="mb-1">Price: {money.format(product.price)}</div>
        <div className="mb-3">Quantity: {product.quantity}</div>
        <div className="mt-auto d-flex gap-2">
          <Button size="sm" variant="warning" onClick={() => onEdit(product)}>
            Edit
          </Button>
          <Button size="sm" variant="danger" onClick={() => onDelete(product.id)}>
            Delete
          </Button>
        </div>
      </Card.Body>
    </Card>
  );
}
```

Bước 8. Tạo ProductList

Mục tiêu: Render collection bằng map() và stable key.

Thao tác

- Tạo ProductList.jsx.
- Nếu array rỗng hiển thị empty state.
- Mỗi Col dùng key={product.id}.

Kết quả mong đợi: Danh sách responsive và không có warning key.

Cách kiểm tra: Console không có Each child in a list should have a unique key.

Lỗi thường gặp: Dùng index làm key dù đã có domain id.

Trạng thái project: V3 – List rendering ready

src/components/ProductList.jsx

```
import Col from "react-bootstrap/Col";
import Row from "react-bootstrap/Row";
import Alert from "react-bootstrap/Alert";
import ProductItem from "../ProductItem";

export default function ProductList({ products, onEdit, onDelete }) {
  if (products.length === 0) {
    return <Alert variant="secondary">No products match the current filter.</Alert>;
  }

  return (
    <Row className="g-3">
      {products.map((product) => (
        <Col key={product.id} xs={12} md={6} xl={4}>
          <ProductItem product={product} onEdit={onEdit} onDelete={onDelete} />
        </Col>
      ))}
    </Row>
  );
}
```

Bước 9. Đưa products vào App bằng useLocalStorage

Mục tiêu: Xác định App là owner của collection state.

Thao tác

- Trong App import initialProducts và useLocalStorage.
- Khai báo `const [products, setProducts] = useLocalStorage(...)`.
- Tạm render ProductList với products.

Kết quả mong đợi: 5 product cards hiển thị từ state được persistent.

Cách kiểm tra: Reload trang vẫn hiển thị 5 record; DevTools thấy key sba301-products.

Lỗi thường gặp: localStorage chứa dữ liệu cũ sai schema: xóa key để reset.

Trạng thái project: V4 – Product collection state

Đoạn state ownership trong App.jsx

```
const [products, setProducts] = useLocalStorage(
  "sba301-products",
  initialProducts
);
```

Bước 10. Tạo ProductForm controlled + validation

Mục tiêu: Thực hành controlled form, object state, effect đồng bộ editingProduct và validation.

Thao tác

- Tạo ProductForm.jsx bằng code đầy đủ.
- Mỗi input lấy value từ form state.

- handleChange cập nhật immutable bằng spread.
- validate trả object lỗi.
- useEffect nạp dữ liệu khi editingProduct thay đổi.

Kết quả mong đợi: Form hiển thị 4 field; submit rỗng báo lỗi; không crash.

Cách kiểm tra: Thử name rỗng, price=0, quantity=-1, category chưa chọn.

Lỗi thường gặp: Input có value nhưng thiếu onChange sẽ bị read-only; mutation form trực tiếp không re-render đúng.

Trạng thái project: V4 – Controlled form + validation

src/components/ProductForm.jsx

```
import { useEffect, useState } from "react";
import Button from "react-bootstrap/Button";
import Card from "react-bootstrap/Card";
import Col from "react-bootstrap/Col";
import Form from "react-bootstrap/Form";
import Row from "react-bootstrap/Row";

const EMPTY_FORM = {
  name: "",
  category: "",
  price: "",
  quantity: ""
};

export default function ProductForm({ editingProduct, onSave, onCancel }) {
  const [form, setForm] = useState(EMPTY_FORM);
  const [errors, setErrors] = useState({});

  useEffect(() => {
    if (editingProduct) {
      setForm({
        name: editingProduct.name,
        category: editingProduct.category,
        price: String(editingProduct.price),
        quantity: String(editingProduct.quantity)
      });
    } else {
      setForm(EMPTY_FORM);
    }
    setErrors({});
  }, [editingProduct]);

  const handleChange = (event) => {
    const { name, value } = event.target;
    setForm((current) => ({ ...current, [name]: value }));
  };

  const validate = () => {
    const nextErrors = {};
    if (!form.name.trim()) nextErrors.name = "Product name is required.";
    if (!form.category) nextErrors.category = "Category is required.";
    if (Number(form.price) <= 0) nextErrors.price = "Price must be greater than 0.";
    if (Number(form.quantity) < 0) nextErrors.quantity = "Quantity must be 0 or greater.";
    return nextErrors;
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    const nextErrors = validate();
    setErrors(nextErrors);
    if (Object.keys(nextErrors).length > 0) return;

    onSave({
```

```

    id: editingProduct?.id,
    name: form.name.trim(),
    category: form.category,
    price: Number(form.price),
    quantity: Number(form.quantity)
  });

  if (!editingProduct) setForm(EMPTY_FORM);
};

return (
  <Card className="shadow-sm mb-4">
    <Card.Body>
      <Card.Title>{editingProduct ? "Edit product" : "Add product"}</Card.Title>
      <Form onSubmit={handleSubmit} noValidate>
        <Row className="g-3">
          <Col md={6}>
            <Form.Group>
              <Form.Label>Name</Form.Label>
              <Form.Control
                name="name"
                value={form.name}
                onChange={handleChange}
                isValid={!Boolean(errors.name)}
              />
              <Form.Control.Feedback type="invalid">{errors.name}</Form.Control.Feedback>
            </Form.Group>
          </Col>
          <Col md={6}>
            <Form.Group>
              <Form.Label>Category</Form.Label>
              <Form.Select
                name="category"
                value={form.category}
                onChange={handleChange}
                isValid={!Boolean(errors.category)}
              >
                <option value="">Choose...</option>
                <option>Accessories</option>
                <option>Display</option>
                <option>Connectivity</option>
                <option>Workspace</option>
                <option>Other</option>
              </Form.Select>
              <Form.Control.Feedback type="invalid">{errors.category}</Form.Control.Feedback>
            </Form.Group>
          </Col>
          <Col md={6}>
            <Form.Group>
              <Form.Label>Price (VND)</Form.Label>
              <Form.Control
                type="number"
                min="1"
                name="price"
                value={form.price}
                onChange={handleChange}
                isValid={!Boolean(errors.price)}
              />
              <Form.Control.Feedback type="invalid">{errors.price}</Form.Control.Feedback>
            </Form.Group>
          </Col>
          <Col md={6}>
            <Form.Group>
              <Form.Label>Quantity</Form.Label>
              <Form.Control
                type="number"
                min="0"
                name="quantity"
                value={form.quantity}
                onChange={handleChange}
                isValid={!Boolean(errors.quantity)}
              />
              <Form.Control.Feedback type="invalid">{errors.quantity}</Form.Control.Feedback>
            </Form.Group>
          </Col>
        </Row>
        <div className="d-flex gap-2 mt-3">
          <Button type="submit" variant="primary">
            {editingProduct ? "Update" : "Add product"}
          </Button>
        </div>
      </Form>
    </Card.Body>
  </Card>
);

```

```

        </Button>
        {editingProduct && (
          <Button type="button" variant="secondary" onClick={onCancel}>
            Cancel
          </Button>
        )}
      </div>
    </Form>
  </Card.Body>
</Card>
);
}

```

Bước 11. Implement Add Product

Mục tiêu: Cập nhật array state immutable bằng spread.

Thao tác

- Trong App viết saveProduct.
- Nếu không có id, tạo id Date.now().
- setProducts(current => [...current, newProduct]).

Kết quả mong đợi: Submit hợp lệ tạo card mới và thống kê sau này tăng.

Cách kiểm tra: Thêm Product Test giá 100000 số lượng 2; card xuất hiện ngay.

Lỗi thường gặp: push() vào array cũ rồi set cùng reference; dữ liệu string chưa convert sang Number.

Trạng thái project: V4 – Create works

```

const newProduct = { ...product, id: Date.now() };
setProducts((current) => [...current, newProduct]);

```

Bước 12. Implement Delete Product

Mục tiêu: Cập nhật array immutable bằng filter và xác nhận trước khi xóa.

Thao tác

- Tìm record theo id.
- window.confirm để xác nhận.
- setProducts(current => current.filter(...)).

Kết quả mong đợi: Card bị xóa khỏi UI và localStorage.

Cách kiểm tra: Xóa một item, reload trang, item vẫn không còn.

Lỗi thường gặp: So sánh id sai type; filter điều kiện ngược.

Trạng thái project: V5 – Delete works

```
setProducts((current) =>
  current.filter((item) => item.id !== id)
);
```

Bước 13. Implement Edit Product

Mục tiêu: Kết nối editingProduct, form prefill và immutable update bằng map.

Thao tác

- App có const [editingProduct, setEditingProduct] = useState(null).
- Edit button truyền product lên App.
- ProductForm nhận editingProduct.
- saveProduct dùng map để thay đúng record.
- Cancel đặt editingProduct về null.

Kết quả mong đợi: Bấm Edit thấy form có dữ liệu cũ; Update thay đúng card, id giữ nguyên.

Cách kiểm tra: Sửa giá một sản phẩm; reload; giá mới còn.

Lỗi thường gặp: Tạo id mới khi sửa; mutate product object trực tiếp; form không reset.

Trạng thái project: V5 – Full CRUD frontend

```
setProducts((current) =>
  current.map((item) =>
    item.id === product.id ? product : item
  )
);
```

Bước 14. Tạo SearchBox và useRef focus

Mục tiêu: Thực hành ref tới DOM input và state search.

Thao tác

- Tạo SearchBox.jsx.
- App khai báo searchTerm useState và searchInputRef useRef.
- Truyền inputRef xuống SearchBox.
- Effect [] focus khi app mount.
- Button Focus gọi inputRef.current?.focus().

Kết quả mong đợi: Cursor nằm trong ô search khi trang mở; nút Focus hoạt động.

Cách kiểm tra: Reload và quan sát focus; click chỗ khác rồi nhấn Focus.

Lỗi thường gặp: Dùng ref.current thay state để điều khiển text hiển thị; ref chưa gắn vào input.

Trạng thái project: V6 – useRef demonstrated

src/components/SearchBox.jsx

```
import Button from "react-bootstrap/Button";
import Form from "react-bootstrap/Form";
import InputGroup from "react-bootstrap/InputGroup";

export default function SearchBox({ value, onChange, inputRef }) {
  return (
    <InputGroup>
      <Form.Control
        ref={inputRef}
        value={value}
        onChange={(event) => onChange(event.target.value)}
        placeholder="Search by product name..."
        aria-label="Search products"
      />
      <Button variant="outline-secondary" onClick={() => inputRef.current?.focus()}>
        Focus
      </Button>
    </InputGroup>
  );
}
```

App.jsx – state + ref + focus effect

```
const [searchTerm, setSearchTerm] = useState("");
const searchInputRef = useRef(null);

useEffect(() => {
  searchInputRef.current?.focus();
}, []);
```

Bước 15. Tạo CategoryFilter

Mục tiêu: Thêm state category và dropdown điều khiển.

Thao tác

- Tạo CategoryFilter.jsx.
- App khai báo category = All.
- Danh sách categories được suy ra từ products.

Kết quả mong đợi: Dropdown hiển thị All và các category có thật.

Cách kiểm tra: Thêm product category Other; dropdown có Other.

Lỗi thường gặp: Lưu categories thành state riêng làm trùng source of truth.

Trạng thái project: V6 – Filter input ready

src/components/CategoryFilter.jsx

```
import Form from "react-bootstrap/Form";

export default function CategoryFilter({ value, categories, onChange }) {
  return (
    <Form.Select value={value} onChange={(event) => onChange(event.target.value)}>
      {categories.map((category) => (
        <option key={category} value={category}>
          {category}
        </option>
      ))}
    </Form.Select>
  );
}
```

```

    )}}
  </Form.Select>
  );
}

```

Bước 16. Tạo derived filtering

Mục tiêu: Không lưu filteredProducts thành state; tính trực tiếp từ products/search/category.

Thao tác

- Chuẩn hóa searchTerm về lowercase.
- Dùng products.filter.
- ProductList nhận filteredProducts.

Kết quả mong đợi: Tìm kiếm và lọc kết hợp; products gốc không bị thay đổi.

Cách kiểm tra: Search Mouse; chọn category không phù hợp thì empty state.

Lỗi thường gặp: Tạo useEffect chỉ để set filtered state; mutation products trong filter.

Trạng thái project: V6 – Search/filter working

```

const normalizedSearch = searchTerm.trim().toLowerCase();
const filteredProducts = products.filter((product) => {
  const matchesName = product.name.toLowerCase().includes(normalizedSearch);
  const matchesCategory = category === "All" || product.category === category;
  return matchesName && matchesCategory;
});

```

Bước 17. Tạo ProductStats

Mục tiêu: Thực hành derived data bằng reduce thay vì thêm state không cần thiết.

Thao tác

- Tạo ProductStats.jsx.
- Tính product types, total quantity, inventory value.
- Render 3 Card.

Kết quả mong đợi: Thống kê thay đổi ngay khi CRUD.

Cách kiểm tra: Thêm/xóa/sửa quantity và đổi chiều số liệu.

Lỗi thường gặp: Lưu totalQuantity bằng useState khiến phải đồng bộ thủ công.

Trạng thái project: V6 – Derived statistics

src/components/ProductStats.jsx

```

import Card from "react-bootstrap/Card";

```

```
import Col from "react-bootstrap/Col";
import Row from "react-bootstrap/Row";

const money = new Intl.NumberFormat("vi-VN", {
  style: "currency",
  currency: "VND"
});

export default function ProductStats({ products }) {
  const itemTypes = products.length;
  const totalQuantity = products.reduce((sum, item) => sum + item.quantity, 0);
  const inventoryValue = products.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  const stats = [
    ["Product types", itemTypes],
    ["Total quantity", totalQuantity],
    ["Inventory value", money.format(inventoryValue)]
  ];

  return (
    <Row className="g-3 mb-4">
      {stats.map(([label, value]) => (
        <Col key={label} xs={12} md={4}>
          <Card className="text-center shadow-sm">
            <Card.Body>
              <div className="text-secondary small">{label}</div>
              <strong className="fs-5">{value}</strong>
            </Card.Body>
          </Card>
        </Col>
      ))}
    </Row>
  );
}
```

Bước 18. useEffect cập nhật document.title

Mục tiêu: Đồng bộ React state với external browser document.

Thao tác

- Thêm effect phụ thuộc products.length.

Kết quả mong đợi: Tab title hiển thị Products (N) | SBA301 và thay đổi sau add/delete.

Cách kiểm tra: Quan sát browser tab sau CRUD.

Lỗi thường gặp: Bỏ dependency khiến title có thể stale; dùng effect cho calculation thuần không cần thiết.

Trạng thái project: V7 – Side effect demonstrated

```
useEffect(() => {
  document.title = `Products (${products.length}) | SBA301`;
}, [products.length]);
```

Bước 19. Feedback effect + cleanup

Mục tiêu: Thực hành timer side effect và cleanup function.

Thao tác

- App có message state.
- Sau CRUD set message.
- Effect tạo setTimeout 2500ms.
- Cleanup clearTimeout khi message đổi/unmount.

Kết quả mong đợi: Alert thành công tự ẩn; timer cũ được cleanup.

Cách kiểm tra: Thực hiện CRUD liên tục và đảm bảo không xuất hiện hành vi alert bất thường.

Lỗi thường gặp: Quên cleanup; setTimeout tạo ở render body; dependency sai.

Trạng thái project: V7 – Cleanup demonstrated

```
useEffect(() => {
  if (!message) return undefined;
  const timerId = setTimeout(() => setMessage(""), 2500);
  return () => clearTimeout(timerId);
}, [message]);
```

Bước 20. Hoàn thiện App, theme và responsive CSS

Mục tiêu: Tích hợp toàn bộ components và Context thành application hoàn chỉnh.

Thao tác

- Thay App.jsx bằng phiên bản đầy đủ.
- Đảm bảo main.jsx bọc ThemeProvider.
- Dùng data-bs-theme={theme}.
- Áp dụng app.css.

Kết quả mong đợi: Tất cả chức năng core cùng hoạt động; dark/light hiển thị rõ.

Cách kiểm tra: Test desktop và mobile width; toggle theme; CRUD; search/filter.

Lỗi thường gặp: App nằm ngoài ThemeProvider; thiếu Bootstrap CSS; import path sai.

Trạng thái project: V8 – Integrated application

src/App.jsx – bản hoàn chỉnh

```
import { useEffect, useRef, useState } from "react";
import Alert from "react-bootstrap/Alert";
import Col from "react-bootstrap/Col";
import Container from "react-bootstrap/Container";
import Row from "react-bootstrap/Row";
import Header from "../components/Header";
import ProductForm from "../components/ProductForm";
import ProductList from "../components/ProductList";
import SearchBox from "../components/SearchBox";
import CategoryFilter from "../components/CategoryFilter";
import ProductStats from "../components/ProductStats";
```

```

import { useTheme } from "../context/ThemeContext";
import { initialProducts } from "../data/initialProducts";
import useLocalStorage from "../hooks/useLocalStorage";

export default function App() {
  const [products, setProducts] = useLocalStorage("sba301-products", initialProducts);
  const [editingProduct, setEditingProduct] = useState(null);
  const [searchTerm, setSearchTerm] = useState("");
  const [category, setCategory] = useState("All");
  const [message, setMessage] = useState("");
  const searchInputRef = useRef(null);
  const { theme } = useTheme();

  useEffect(() => {
    document.title = `Products (${products.length}) | SBA301`;
  }, [products.length]);

  useEffect(() => {
    searchInputRef.current?.focus();
  }, []);

  useEffect(() => {
    if (!message) return undefined;
    const timerId = setTimeout(() => setMessage(""), 2500);
    return () => clearTimeout(timerId);
  }, [message]);

  const saveProduct = (product) => {
    if (product.id) {
      setProducts((current) =>
        current.map((item) => (item.id === product.id ? product : item))
      );
      setMessage("Product updated successfully.");
      setEditingProduct(null);
      return;
    }

    const newProduct = { ...product, id: Date.now() };
    setProducts((current) => [...current, newProduct]);
    setMessage("Product added successfully.");
  };

  const deleteProduct = (id) => {
    const product = products.find((item) => item.id === id);
    if (!product) return;
    if (!window.confirm(`Delete ${product.name}?`)) return;
    setProducts((current) => current.filter((item) => item.id !== id));
    if (editingProduct?.id === id) setEditingProduct(null);
    setMessage("Product deleted successfully.");
  };

  const categories = ["All", ...new Set(products.map((item) => item.category))];
  const normalizedSearch = searchTerm.trim().toLowerCase();
  const filteredProducts = products.filter((product) => {
    const matchesName = product.name.toLowerCase().includes(normalizedSearch);
    const matchesCategory = category === "All" || product.category === category;
    return matchesName && matchesCategory;
  });

  return (
    <div className="min-vh-100 app-shell" data-bs-theme={theme}>
      <Header />
      <Container className="py-4">
        <div className="mb-4">
          <h1 className="h3 mb-1">Product Management Dashboard</h1>
          <p className="text-secondary mb-0">
            Frontend CRUD + React Hooks + localStorage
          </p>
        </div>

        {message && <Alert variant="success">{message}</Alert>}

        <ProductStats products={products} />

        <ProductForm

```

```

        editingProduct={editingProduct}
        onSave={saveProduct}
        onCancel={() => setEditingProduct(null)}
      />

      <Row className="g-3 mb-4">
        <Col md={8}>
          <SearchBox
            value={searchTerm}
            onChange={setSearchTerm}
            inputRef={searchInputRef}
          />
        </Col>
        <Col md={4}>
          <CategoryFilter
            value={category}
            categories={categories}
            onChange={setCategory}
          />
        </Col>
      </Row>

      <ProductList
        products={filteredProducts}
        onEdit={setEditingProduct}
        onDelete={deleteProduct}
      />
    </Container>
  </div>
);
}

```

src/main.jsx – bản hoàn chỉnh

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "./styles/app.css";
import App from "./App";
import { ThemeProvider } from "./context/ThemeContext";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </StrictMode>
);

```

src/styles/app.css – bản hoàn chỉnh

```

html, body, #root { min-height: 100%; }
body { margin: 0; }
.app-shell { background: #f6f7f9; transition: background-color .2s ease; }
.app-shell[data-bs-theme="dark"] { background: #17191c; }
.card { border-radius: 0.8rem; }
.navbar-brand { font-weight: 700; }
.form-control, .form-select, .btn { border-radius: 0.6rem; }
@media (max-width: 576px) {
  .navbar-brand { font-size: 1rem; }
}

```

Bước 21. Kiểm tra Rules of Hooks + localStorage

Mục tiêu: Xác minh code đúng quy tắc Hook và persistence thực sự hoạt động.

Thao tác

- Search toàn project useState/useEffect/useContext/useRef.
- Xác nhận Hook không nằm trong if/loop/handler.

- DevTools > Application > Local Storage kiểm tra sba301-products và sba301-theme.
- Reload trang.

Kết quả mong đợi: Không có Hook rule violation; dữ liệu và theme giữ sau reload.

Cách kiểm tra: F5 không mất products/theme; console không có error.

Lỗi thường gặp: Nhầm Hook với hàm thông thường; gọi useTheme trong function không phải component/hook.

Trạng thái project: V8 – Hook compliance

Bước 22. Git checkpoint và production build

Mục tiêu: Tạo evidence version và xác minh source có thể build.

Thao tác

- Chạy git status.
- git add .
- commit.
- npm run build.
- npm run preview để kiểm tra build cục bộ.

Kết quả mong đợi: Commit tồn tại; dist được tạo; preview chạy.

Cách kiểm tra: git log -1 và terminal build không lỗi.

Lỗi thường gặp: Commit khi project đang FAIL; build fail do import case-sensitive.

Trạng thái project: V8 – Build verified

```
git status
git add .
git commit -m "Complete Slot 6 React Hook Product Manager"
git log -1
npm run build
npm run preview
```

Bước 23. Final Verification Gate

Mục tiêu: Chỉ kết luận hoàn thành khi chức năng, Hook, console và evidence đều PASS.

Thao tác

- Chạy test matrix ở Phần 9.
- Chụp ảnh app light/dark, CRUD, localStorage, build.
- Điền checklist Slot 6.
- Không dùng AI trong phần giải thích Human Gate.

Kết quả mong đợi: Không lỗi blocking; 100% core test cases PASS hoặc lỗi được ghi rõ.

Cách kiểm tra: Sinh viên giải thích được ownership của products/editingProduct/search/category/theme/ref và lý do từng Hook.

Lỗi thường gặp: Chỉ nhìn giao diện đẹp mà không kiểm tra console/data flow.

Trạng thái project: FINAL – Ready to submit

8. ĐÁNH GIÁ KẾT QUẢ HOÀN THÀNH TỪNG BƯỚC

Bước	Nội dung	Kết quả mong đợi	Cách kiểm tra	Trạng thái
0	Baseline môi trường và project	Vite app chạy	Browser/Console/Code evidence	☐ PASS ☐ FAIL
1	Dọn code mặc định, import Bootstrap/CSS	Bootstrap/CSS load	Browser/Console/Code evidence	☐ PASS ☐ FAIL
2	Tạo cấu trúc thư mục	Folder đúng	Browser/Console/Code evidence	☐ PASS ☐ FAIL
3	Tạo dữ liệu initialProducts	5 mock records	Browser/Console/Code evidence	☐ PASS ☐ FAIL
4	Tạo custom hook useLocalStorage	Custom hook tồn tại	Browser/Console/Code evidence	☐ PASS ☐ FAIL
5	Tạo ThemeContext và useTheme	Theme context hoạt động	Browser/Console/Code evidence	☐ PASS ☐ FAIL
6	Tạo Header + ThemeToggle	Navbar + toggle	Browser/Console/Code evidence	☐ PASS ☐ FAIL
7	Tạo ProductItem	Product card	Browser/Console/Code evidence	☐ PASS ☐ FAIL
8	Tạo ProductList	List + empty state	Browser/Console/Code evidence	☐ PASS ☐ FAIL
9	Đưa products vào App bằng useLocalStorage	Products state persisted	Browser/Console/Code evidence	☐ PASS ☐ FAIL
10	Tạo ProductForm controlled + validation	Form + validation	Browser/Console/Code evidence	☐ PASS ☐ FAIL
11	Thêm sản phẩm	Add works	Browser/Console/Code evidence	☐ PASS ☐ FAIL
12	Xóa sản phẩm	Delete works	Browser/Console/Code evidence	☐ PASS ☐ FAIL
13	Sửa sản phẩm	Edit works	Browser/Console/Code evidence	☐ PASS ☐ FAIL
14	Tạo SearchBox + useRef focus	Focus/search input	Browser/Console/Code evidence	☐ PASS ☐ FAIL
15	Tạo CategoryFilter	Category dropdown	Browser/Console/Code evidence	☐ PASS ☐ FAIL
16	Tạo derived filtering	Combined filter	Browser/Console/Code evidence	☐ PASS ☐ FAIL

Bước	Nội dung	Kết quả mong đợi	Cách kiểm tra	Trạng thái
17	Tạo ProductStats	Stats đúng	Browser/Console/Code evidence	☐ PASS ☐ FAIL
18	Thêm useEffect cập nhật document.title	Title effect	Browser/Console/Code evidence	☐ PASS ☐ FAIL
19	Thêm feedback effect + cleanup	Cleanup timer	Browser/Console/Code evidence	☐ PASS ☐ FAIL
20	Hoàn thiện CSS/theme/responsive	Full app integrated	Browser/Console/Code evidence	☐ PASS ☐ FAIL
21	Kiểm tra Rules of Hooks + localStorage	Hooks/localStorage PASS	Browser/Console/Code evidence	☐ PASS ☐ FAIL
22	Git checkpoint + build	Git/build PASS	Browser/Console/Code evidence	☐ PASS ☐ FAIL
23	Final Verification Gate	Final tests PASS	Browser/Console/Code evidence	☐ PASS ☐ FAIL

9. CHẠY VÀ KIỂM TRA ỨNG DỤNG HOÀN CHỈNH

```
npm install
npm run dev
```

Mở URL do Vite in ra (thường bắt đầu bằng `http://localhost:5173`). Không giả định port cố định nếu Vite đổi port vì cổng đang bận.

STT	Chức năng	Thao tác	Kết quả mong đợi	Đạt
TC01	Khởi động	npm run dev	App mở, không lỗi blocking	☐
TC02	Dữ liệu đầu	Mở trang lần đầu	5 product cards	☐
TC03	Validation name	Submit name rỗng	Báo Product name is required	☐
TC04	Validation price	Price = 0	Báo price > 0	☐
TC05	Validation qty	Quantity = -1	Báo quantity >= 0	☐
TC06	Add	Nhập record hợp lệ, Add	Record mới xuất hiện	☐
TC07	Edit	Edit record, thay price, Update	Chỉ record đó đổi	☐
TC08	Cancel edit	Edit rồi Cancel	Form về add mode	☐
TC09	Delete cancel	Delete rồi Cancel confirm	Record vẫn còn	☐
TC10	Delete confirm	Delete rồi OK	Record biến mất	☐
TC11	Search	Nhập Mouse	Chỉ record match	☐
TC12	Category	Chọn Display	Chỉ Display	☐
TC13	Combined filter	Search + category	Kết quả giao nhau	☐
TC14	Empty state	Tìm chuỗi không tồn tại	No products match...	☐
TC15	Stats	CRUD quantity/price	Stats cập nhật đúng	☐
TC16	Theme	Toggle dark/light	Theme đổi toàn app	☐
TC17	Persistence products	Add rồi F5	Record còn	☐
TC18	Persistence theme	Dark rồi F5	Theme dark còn	☐
TC19	useRef	Reload / nút Focus	Search input focus	☐

STT	Chức năng	Thao tác	Kết quả mong đợi	Đạt
TC20	Title effect	Add/Delete	Tab title đổi N	☐
TC21	Cleanup behavior	CRUD liên tục	Alert tự ẩn ổn định	☐
TC22	Console	Mở DevTools Console	Không error nghiêm trọng	☐
TC23	Build	npm run build	Build thành công	☐
TC24	Preview	npm run preview	Production preview chạy	☐

9.1 Human Verification Gate

Không dùng AI, sinh viên phải giải thích: products được sở hữu ở đâu; vì sao filteredProducts không cần state; useEffect nào có cleanup và vì sao; ref khác state thế nào; Context được dùng vì sao; custom hook tái sử dụng logic gì; localStorage khác database thế nào.

10. LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Dấu hiệu	Nguyên nhân	Evidence	Cách xử lý	Phòng tránh
npm: package.json not found	Terminal ở sai thư mục	Prompt/path terminal	cd vào project có package.json	Luôn kiểm tra working directory
Vite không chạy với Node 21	Vite hiện hành không support Node 21	node -v + warning npm	Dùng Node 22.12+ hoặc pin Vite 5 theo lớp	Khóa toolchain từ đầu
Bootstrap không có style	Thiếu import CSS	main.jsx	import bootstrap/dist/css/bootstrap.min.css	Kiểm tra entry point
Component blank	Import/export/path lỗi	Console first error	Sửa lỗi import đầu tiên	Không debug CSS trước compile
State không đổi	Mutation trực tiếp hoặc setter sai	React DevTools/code	Dùng setter + immutable update	Không push/splice state array
Effect loop	Effect thay state nằm trong dependency	Console/render behavior	Xem lại dependency và mục đích effect	Không dùng Effect cho derived calculation
Form read-only	Có value nhưng thiếu onChange	Console/UI	Thêm onChange cập nhật state	Controlled input đủ value + onChange
Edit form không nạp	editingProduct/effect mismatch	Props + effect dependencies	Kiểm tra prop và [editingProduct]	Trace parent→child
Context undefined	Ngoài Provider hoặc import khác context	Component tree	Bọc ThemeProvider đúng chỗ	Provider gần root
Theme value đổi nhưng UI không đổi	UI không consume theme	React DevTools	Gắn data-bs-theme/class từ theme	Trace Context→consumer
Ref null	Ref chưa gắn DOM hoặc access quá sớm	input JSX/effect	ref={inputRef}, access after mount/event	Ref không thay UI state
Reload mất dữ liệu	Không ghi localStorage/key sai	DevTools Application	Kiểm tra useLocalStorage effect	Một key ổn định
JSON parse error	Storage cũ/hỏng	Console + stored value	Xóa key/reset; giữ try/catch	Đọc storage defensively
Xóa nhầm item	id/key không ổn định	Data array	Dùng domain id	Không dùng random key mỗi render
Build fail khác dev	Import case mismatch	npm run build output	Sửa đúng chữ hoa/thường	Đặt file/import nhất quán

Debugging Decision Tree: App compile? → đúng working directory/dependencies? → first terminal error? → browser console? → state/props đúng? → event chạy? → setter chạy? → JSX phụ thuộc state? → nếu effect: dependency/cleanup? → nếu context: Provider? → nếu ref: DOM đã gắn?

11. PHẦN MỞ RỘNG VỚI AI ASSISTANT

Nguyên tắc: AI có thể giải thích, review, gợi ý test và hỗ trợ debug; sinh viên phải cung cấp evidence và tự xác minh. Không dùng AI để thay thế Human Verification Gate.

11.1 10 prompt giải thích code

1. Giải thích data ownership của products, editingProduct, searchTerm và category trong App.jsx. Không viết lại code.
2. Trace useState trong ProductForm từ input change → setter → re-render → value hiển thị.
3. Giải thích vì sao products được lưu bằng useLocalStorage nhưng filteredProducts không được lưu bằng state.
4. Giải thích dependency array [products.length] trong effect cập nhật document.title.
5. Giải thích cleanup function của timer feedback và tình huống cleanup được gọi.
6. Giải thích ThemeContext, ThemeProvider và custom hook useTheme theo component tree.
7. Giải thích useRef của searchInputRef và vì sao thay đổi ref.current không cần re-render.
8. Giải thích useLocalStorage từ lazy initial state đến JSON.parse/JSON.stringify và effect synchronization.
9. Trace workflow Add Product từ controlled form đến ProductList và localStorage.
10. Trace workflow Edit Product: ProductItem → App state → ProductForm → immutable map update.

11.2 10 prompt tối ưu code

11. Review component responsibilities của project. Chỉ ra component quá lớn hoặc responsibility bị lặp; không thêm thư viện.
12. Đánh giá state nào thật sự cần useState và state nào nên là derived value.
13. Đánh giá useEffect nào cần thiết và effect nào có thể loại bỏ theo nguyên tắc You Might Not Need an Effect.
14. Đề xuất cách làm validation rõ ràng hơn nhưng vẫn ở mức SBA301 và không dùng form library.
15. Đánh giá useLocalStorage về error handling, initialization và dependency array.
16. Đánh giá Context hiện tại có bị lạm dụng không; chỉ giữ shared concern hợp lý.
17. Đề xuất cải thiện UX Add/Edit/Delete mà không thêm Router/backend.
18. Đánh giá khả năng re-render; chỉ đề xuất useMemo/useCallback nếu có bằng chứng cần thiết.
19. Đề xuất tách service/data layer để chuẩn bị thay localStorage bằng Spring Boot REST sau này, nhưng chưa viết backend.
20. Kiểm tra naming, import/export và folder structure theo tiêu chí readability/maintainability.

11.3 10 prompt tìm hiểu workflow

21. Mô tả workflow từ lúc browser load đến khi initialProducts/localStorage trở thành UI.
22. Mô tả workflow thêm sản phẩm theo chuỗi Event → Validation → Setter → Re-render → Effect → localStorage.
23. Mô tả workflow sửa sản phẩm và state ownership của editingProduct.
24. Mô tả workflow xóa sản phẩm và vì sao filter là immutable update.
25. Mô tả workflow search và vì sao filteredProducts là derived data.
26. Mô tả workflow category filter kết hợp search term.
27. Mô tả workflow theme từ ThemeToggle đến Context Provider rồi đến App data-bs-theme.
28. Mô tả workflow reload trang và việc useLocalStorage phục hồi dữ liệu.
29. Mô tả workflow useRef focus khi mount và khi người dùng nhấn Focus.
30. Nếu thay localStorage bằng Spring Boot API ở slot sau, xác định phần nào giữ nguyên và phần nào phải thay.

11.4 5 prompt debug có evidence

31. Ứng dụng không re-render sau Add. Đây là state code, handler và console. Trace Event → Setter → new reference → JSX, chỉ ra FIRST broken stage.
32. useEffect chạy lặp. Đây là effect và dependency array. Phân tích biến nào thay đổi do chính effect và đề xuất sửa tối thiểu.
33. Theme không đổi. Đây là ThemeProvider, useTheme consumer và App wrapper. Trace Provider value → consumer → rendered attribute.
34. Reload mất dữ liệu. Đây là localStorage key, hook và DevTools stored value. Kiểm tra read → parse → state → write.

35. Edit form không prefill. Đây là editingProduct ở App, props ProductForm và effect. Trace giá trị qua từng bước và tìm mismatch đầu tiên.

11.5 Prompt mở rộng nâng cao

- Thêm sort theo tên/giá nhưng giữ filter là derived data.
- Thêm pagination frontend cho 100 mock products.
- Thêm React-Bootstrap Modal xác nhận xóa thay window.confirm.
- Thêm Toast notification có cleanup đúng.
- Chuyển project sang TypeScript và định nghĩa Product type.
- Thiết kế ProductService interface để sau này thay localStorage bằng Spring Boot REST API.

AI Evidence Log: Với mỗi lần dùng AI, lưu: Prompt → câu trả lời chính → phần code thay đổi → test đã chạy → kết luận chấp nhận/từ chối → bài học rút ra.

12. CHECKLIST Củng cố kiến thức SLOT 6

Nội dung	Có	Không	Cần làm lại
Tôi hiểu Hook là gì và vì sao cần Hook	☐	☐	☐
Tôi dùng useState cho dữ liệu ảnh hưởng UI	☐	☐	☐
Tôi cập nhật object/array immutable	☐	☐	☐
Tôi phân biệt derived data với state	☐	☐	☐
Tôi dùng useEffect cho side effect/synchronization	☐	☐	☐
Tôi hiểu dependency array	☐	☐	☐
Tôi viết được cleanup function	☐	☐	☐
Tôi dùng useContext có chọn lọc	☐	☐	☐
Tôi dùng useRef cho DOM/value không re-render	☐	☐	☐
Tôi tạo được custom hook useLocalStorage	☐	☐	☐
Tôi xây controlled form + validation	☐	☐	☐
Tôi tuân thủ Rules of Hooks	☐	☐	☐
Tôi kiểm tra được localStorage bằng DevTools	☐	☐	☐
Tôi chạy được project trong IntelliJ	☐	☐	☐
Tôi chạy npm run build	☐	☐	☐
Tôi giải thích được data flow không cần AI	☐	☐	☐
Tôi dùng AI theo evidence-driven workflow	☐	☐	☐

13. RUBRIC ĐÁNH GIÁ BÀI TẬP TỔNG HỢP

Tiêu chí	Điểm	Evidence tối thiểu
Project chạy được + build	15	Dev server/build PASS, không lỗi blocking
CRUD frontend	20	Add/Edit/Delete đúng, immutable update
React Hooks	20	useState/useEffect/useContext/useRef + custom hook đúng

Tiêu chí	Điểm	Evidence tối thiểu
Form + validation	10	Controlled form, validation core cases
Architecture	10	Components/context/hooks/data rõ trách nhiệm
Theme Context	5	Provider/consumer/toggle đúng
Persistence/custom hook	10	Reload giữ data/theme, hook tái sử dụng
README/Evidence	5	Cách chạy, screenshot/test/Git evidence
Code quality	5	Naming, readability, no severe warnings

Mức đạt: Đạt: chạy được và đủ core. Khá: validation/custom hook tốt. Tốt: UX, evidence, README rõ. Xuất sắc: mở rộng hợp lý, AI log và test evidence nhưng không overengineer.

PHỤ LỤC A. SOURCE CODE HOÀN CHỈNH

Phụ lục này dùng để đối chiếu sau khi đã thực hành từng bước. Không nên bắt đầu buổi lab bằng cách copy toàn bộ phụ lục.

src/data/initialProducts.js

```
export const initialProducts = [
  { id: 1, name: "Mechanical Keyboard", category: "Accessories", price: 1490000, quantity: 8 },
  { id: 2, name: "Wireless Mouse", category: "Accessories", price: 690000, quantity: 15 },
  { id: 3, name: "27-inch Monitor", category: "Display", price: 5290000, quantity: 4 },
  { id: 4, name: "USB-C Hub", category: "Connectivity", price: 890000, quantity: 11 },
  { id: 5, name: "Laptop Stand", category: "Workspace", price: 790000, quantity: 9 }
];
```

src/hooks/useLocalStorage.js

```
import { useEffect, useState } from "react";

export default function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const storedValue = localStorage.getItem(key);
      return storedValue !== null ? JSON.parse(storedValue) : initialValue;
    } catch (error) {
      console.error(`Cannot read localStorage key: ${key}`, error);
      return initialValue;
    }
  });

  useEffect(() => {
    try {
      localStorage.setItem(key, JSON.stringify(value));
    } catch (error) {
      console.error(`Cannot write localStorage key: ${key}`, error);
    }
  }, [key, value]);

  return [value, setValue];
}
```

src/context/ThemeContext.jsx

```
import { createContext, useContext } from "react";
import useLocalStorage from "../hooks/useLocalStorage";
```

```

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useLocalStorage("sba301-theme", "light");

  const toggleTheme = () => {
    setTheme((current) => (current === "light" ? "dark" : "light"));
  };

  const value = { theme, toggleTheme };

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme must be used inside ThemeProvider");
  }
  return context;
}

```

src/components/ThemeToggle.jsx

```

import Button from "react-bootstrap/Button";
import { useTheme } from "../context/ThemeContext";

export default function ThemeToggle() {
  const { theme, toggleTheme } = useTheme();

  return (
    <Button variant="outline-light" size="sm" onClick={toggleTheme}>
      {theme === "light" ? "Dark mode" : "Light mode"}
    </Button>
  );
}

```

src/components/Header.jsx

```

import Container from "react-bootstrap/Container";
import Navbar from "react-bootstrap/Navbar";
import ThemeToggle from "../ThemeToggle";

export default function Header() {
  return (
    <Navbar bg="primary" data-bs-theme="dark" className="shadow-sm">
      <Container>
        <Navbar.Brand>React Hook Product Manager</Navbar.Brand>
        <ThemeToggle />
      </Container>
    </Navbar>
  );
}

```

src/components/ProductItem.jsx

```

import Button from "react-bootstrap/Button";
import Card from "react-bootstrap/Card";

const money = new Intl.NumberFormat("vi-VN", {
  style: "currency",
  currency: "VND"
});

```

```

export default function ProductItem({ product, onEdit, onDelete }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Body className="d-flex flex-column">
        <Card.Title>{product.name}</Card.Title>
        <Card.Subtitle className="mb-2 text-secondary">
          {product.category}
        </Card.Subtitle>
        <div className="mb-1">Price: {money.format(product.price)}</div>
        <div className="mb-3">Quantity: {product.quantity}</div>
        <div className="mt-auto d-flex gap-2">
          <Button size="sm" variant="warning" onClick={() => onEdit(product)}>
            Edit
          </Button>
          <Button size="sm" variant="danger" onClick={() => onDelete(product.id)}>
            Delete
          </Button>
        </div>
      </Card.Body>
    </Card>
  );
}

```

src/components/ProductList.jsx

```

import Col from "react-bootstrap/Col";
import Row from "react-bootstrap/Row";
import Alert from "react-bootstrap/Alert";
import ProductItem from "../ProductItem";

export default function ProductList({ products, onEdit, onDelete }) {
  if (products.length === 0) {
    return <Alert variant="secondary">No products match the current filter.</Alert>;
  }

  return (
    <Row className="g-3">
      {products.map((product) => (
        <Col key={product.id} xs={12} md={6} xl={4}>
          <ProductItem product={product} onEdit={onEdit} onDelete={onDelete} />
        </Col>
      ))}
    </Row>
  );
}

```

src/components/SearchBox.jsx

```

import Button from "react-bootstrap/Button";
import Form from "react-bootstrap/Form";
import InputGroup from "react-bootstrap/InputGroup";

export default function SearchBox({ value, onChange, inputRef }) {
  return (
    <InputGroup>
      <Form.Control
        ref={inputRef}
        value={value}
        onChange={(event) => onChange(event.target.value)}
        placeholder="Search by product name..."
        aria-label="Search products"
      />
      <Button variant="outline-secondary" onClick={() => inputRef.current?.focus()}>
        Focus
      </Button>
    </InputGroup>
  );
}

```

src/components/CategoryFilter.jsx

```
import Form from "react-bootstrap/Form";

export default function CategoryFilter({ value, categories, onChange }) {
  return (
    <Form.Select value={value} onChange={(event) => onChange(event.target.value)}>
      {categories.map((category) => (
        <option key={category} value={category}>
          {category}
        </option>
      ))}
    </Form.Select>
  );
}
```

src/components/ProductStats.jsx

```
import Card from "react-bootstrap/Card";
import Col from "react-bootstrap/Col";
import Row from "react-bootstrap/Row";

const money = new Intl.NumberFormat("vi-VN", {
  style: "currency",
  currency: "VND"
});

export default function ProductStats({ products }) {
  const itemTypes = products.length;
  const totalQuantity = products.reduce((sum, item) => sum + item.quantity, 0);
  const inventoryValue = products.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  const stats = [
    ["Product types", itemTypes],
    ["Total quantity", totalQuantity],
    ["Inventory value", money.format(inventoryValue)]
  ];

  return (
    <Row className="g-3 mb-4">
      {stats.map(([label, value]) => (
        <Col key={label} xs={12} md={4}>
          <Card className="text-center shadow-sm">
            <Card.Body>
              <div className="text-secondary small">{label}</div>
              <strong className="fs-5">{value}</strong>
            </Card.Body>
          </Card>
        </Col>
      ))}
    </Row>
  );
}
```

src/components/ProductForm.jsx

```
import { useEffect, useState } from "react";
import Button from "react-bootstrap/Button";
import Card from "react-bootstrap/Card";
import Col from "react-bootstrap/Col";
import Form from "react-bootstrap/Form";
import Row from "react-bootstrap/Row";

const EMPTY_FORM = {
  name: "",
  category: "",
  price: "",
  quantity: ""
}
```

```

    });

export default function ProductForm({ editingProduct, onSave, onCancel }) {
  const [form, setForm] = useState(EMPTY_FORM);
  const [errors, setErrors] = useState({});

  useEffect(() => {
    if (editingProduct) {
      setForm({
        name: editingProduct.name,
        category: editingProduct.category,
        price: String(editingProduct.price),
        quantity: String(editingProduct.quantity)
      });
    } else {
      setForm(EMPTY_FORM);
    }
    setErrors({});
  }, [editingProduct]);

  const handleChange = (event) => {
    const { name, value } = event.target;
    setForm((current) => ({ ...current, [name]: value }));
  };

  const validate = () => {
    const nextErrors = {};
    if (!form.name.trim()) nextErrors.name = "Product name is required.";
    if (!form.category) nextErrors.category = "Category is required.";
    if (Number(form.price) <= 0) nextErrors.price = "Price must be greater than 0.";
    if (Number(form.quantity) < 0) nextErrors.quantity = "Quantity must be 0 or greater.";
    return nextErrors;
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    const nextErrors = validate();
    setErrors(nextErrors);
    if (Object.keys(nextErrors).length > 0) return;

    onSave({
      id: editingProduct?.id,
      name: form.name.trim(),
      category: form.category,
      price: Number(form.price),
      quantity: Number(form.quantity)
    });

    if (!editingProduct) setForm(EMPTY_FORM);
  };

  return (
    <Card className="shadow-sm mb-4">
      <Card.Body>
        <Card.Title>{editingProduct ? "Edit product" : "Add product"}</Card.Title>
        <Form onSubmit={handleSubmit} noValidate>
          <Row className="g-3">
            <Col md={6}>
              <Form.Group>
                <Form.Label>Name</Form.Label>
                <Form.Control
                  name="name"
                  value={form.name}
                  onChange={handleChange}
                  isValid={!Boolean(errors.name)}
                />
                <Form.Control.Feedback type="invalid">{errors.name}</Form.Control.Feedback>
              </Form.Group>
            </Col>
            <Col md={6}>
              <Form.Group>
                <Form.Label>Category</Form.Label>
                <Form.Select
                  name="category"
                  value={form.category}
                  onChange={handleChange}
                />
              </Form.Group>
            </Col>
          </Row>
        </Form>
      </Card.Body>
    </Card>
  );
}

```

```

        isValid={Boolean(errors.category)}
      >
        <option value="">Choose...</option>
        <option>Accessories</option>
        <option>Display</option>
        <option>Connectivity</option>
        <option>Workspace</option>
        <option>Other</option>
      </Form.Select>
      <Form.Control.Feedback type="invalid">{errors.category}</Form.Control.Feedback>
    </Form.Group>
  </Col>
  <Col md={6}>
    <Form.Group>
      <Form.Label>Price (VND)</Form.Label>
      <Form.Control
        type="number"
        min="1"
        name="price"
        value={form.price}
        onChange={handleChange}
        isValid={Boolean(errors.price)}
      />
      <Form.Control.Feedback type="invalid">{errors.price}</Form.Control.Feedback>
    </Form.Group>
  </Col>
  <Col md={6}>
    <Form.Group>
      <Form.Label>Quantity</Form.Label>
      <Form.Control
        type="number"
        min="0"
        name="quantity"
        value={form.quantity}
        onChange={handleChange}
        isValid={Boolean(errors.quantity)}
      />
      <Form.Control.Feedback type="invalid">{errors.quantity}</Form.Control.Feedback>
    </Form.Group>
  </Col>
</Row>
<div className="d-flex gap-2 mt-3">
  <Button type="submit" variant="primary">
    {editingProduct ? "Update" : "Add product"}
  </Button>
  {editingProduct && (
    <Button type="button" variant="secondary" onClick={onCancel}>
      Cancel
    </Button>
  )}
</div>
</Form>
</Card.Body>
</Card>
);
}

```

src/App.jsx

```

import { useEffect, useRef, useState } from "react";
import Alert from "react-bootstrap/Alert";
import Col from "react-bootstrap/Col";
import Container from "react-bootstrap/Container";
import Row from "react-bootstrap/Row";
import Header from "../components/Header";
import ProductForm from "../components/ProductForm";
import ProductList from "../components/ProductList";
import SearchBox from "../components/SearchBox";
import CategoryFilter from "../components/CategoryFilter";
import ProductStats from "../components/ProductStats";
import { useTheme } from "../context/ThemeContext";
import { initialProducts } from "../data/initialProducts";
import useLocalStorage from "../hooks/useLocalStorage";

export default function App() {
  const [products, setProducts] = useLocalStorage("sba301-products", initialProducts);

```

```

const [editingProduct, setEditingProduct] = useState(null);
const [searchTerm, setSearchTerm] = useState("");
const [category, setCategory] = useState("All");
const [message, setMessage] = useState("");
const searchInputRef = useRef(null);
const { theme } = useTheme();

useEffect(() => {
  document.title = `Products (${products.length}) | SBA301`;
}, [products.length]);

useEffect(() => {
  searchInputRef.current?.focus();
}, []);

useEffect(() => {
  if (!message) return undefined;
  const timerId = setTimeout(() => setMessage(""), 2500);
  return () => clearTimeout(timerId);
}, [message]);

const saveProduct = (product) => {
  if (product.id) {
    setProducts((current) =>
      current.map((item) => (item.id === product.id ? product : item))
    );
    setMessage("Product updated successfully.");
    setEditingProduct(null);
    return;
  }

  const newProduct = { ...product, id: Date.now() };
  setProducts((current) => [...current, newProduct]);
  setMessage("Product added successfully.");
};

const deleteProduct = (id) => {
  const product = products.find((item) => item.id === id);
  if (!product) return;
  if (!window.confirm(`Delete ${product.name}?`)) return;
  setProducts((current) => current.filter((item) => item.id !== id));
  if (editingProduct?.id === id) setEditingProduct(null);
  setMessage("Product deleted successfully.");
};

const categories = ["All", ...new Set(products.map((item) => item.category))];
const normalizedSearch = searchTerm.trim().toLowerCase();
const filteredProducts = products.filter((product) => {
  const matchesName = product.name.toLowerCase().includes(normalizedSearch);
  const matchesCategory = category === "All" || product.category === category;
  return matchesName && matchesCategory;
});

return (
  <div className="min-vh-100 app-shell" data-bs-theme={theme}>
    <Header />
    <Container className="py-4">
      <div className="mb-4">
        <h1 className="h3 mb-1">Product Management Dashboard</h1>
        <p className="text-secondary mb-0">
          Frontend CRUD + React Hooks + localStorage
        </p>
      </div>

      {message && <Alert variant="success">{message}</Alert>}

      <ProductStats products={products} />

      <ProductForm
        editingProduct={editingProduct}
        onSave={saveProduct}
        onCancel={() => setEditingProduct(null)}
      />

      <Row className="g-3 mb-4">

```

```

        <Col md={8}>
          <SearchBox
            value={searchTerm}
            onChange={setSearchTerm}
            inputRef={searchInputRef}
          />
        </Col>
        <Col md={4}>
          <CategoryFilter
            value={category}
            categories={categories}
            onChange={setCategory}
          />
        </Col>
      </Row>

      <ProductList
        products={filteredProducts}
        onEdit={setEditingProduct}
        onDelete={deleteProduct}
      />
    </Container>
  </div>
);
}

```

src/main.jsx

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "bootstrap/dist/css/bootstrap.min.css";
import "./styles/app.css";
import App from "./App";
import { ThemeProvider } from "./context/ThemeContext";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </StrictMode>
);

```

src/styles/app.css

```

html, body, #root { min-height: 100%; }
body { margin: 0; }
.app-shell { background: #f6f7f9; transition: background-color .2s ease; }
.app-shell[data-bs-theme="dark"] { background: #17191c; }
.card { border-radius: 0.8rem; }
.navbar-brand { font-weight: 700; }
.form-control, .form-select, .btn { border-radius: 0.6rem; }
@media (max-width: 576px) {
  .navbar-brand { font-size: 1rem; }
}

```

PHỤ LỤC B. README MẪU & EVIDENCE PACKAGE

README.md mẫu

```

# React Hook Product Manager

## Course
SBA301 - Slot 6 - React Hook

## Features
- Frontend CRUD: add, edit, delete
- Search and category filter
- Product statistics

```

```

- Dark/light theme
- localStorage persistence
- Auto-focus search input
- document.title effect

## Hooks demonstrated
- useState
- useEffect
- useContext
- useRef
- Custom Hooks: useLocalStorage, useTheme

## Run
npm install
npm run dev

## Build
npm run build
npm run preview

## Evidence
Include screenshots of CRUD, theme, localStorage, console and final build.

```

Evidence Package

Tạo thư mục evidence/ hoặc nộp theo LMS: 01-baseline.png, 02-crud.png, 03-search-filter.png, 04-theme.png, 05-localstorage.png, 06-console.png, 07-build.txt, 08-git-log.txt, AI-log.md nếu có.

PHỤ LỤC C. QUICK REFERENCE

Câu hỏi	Chọn
Dữ liệu thay đổi và cần UI cập nhật?	useState
Cần đồng bộ với external system/timer/storage/document?	useEffect
Dữ liệu dùng chung qua nhiều descendant và props trở nên bất tiện?	Context + useContext
Cần tham chiếu DOM hoặc lưu mutable value không re-render?	useRef
Logic Hook lặp giữa nhiều component?	Custom Hook
Giá trị tính được từ props/state hiện tại?	Derived value, thường không cần state
Cần lưu bền trong browser?	localStorage; có thể bọc trong custom Hook
Cần dữ liệu từ server?	Không thuộc core Slot 6; chuyển sang HTTP/REST ở slot sau

Quick commands

```

# Dev
npm run dev

# Production check
npm run build
npm run preview

# Git
git status
git add .
git commit -m "Complete Slot 6 React Hook Product Manager"
git log -1

```

TÀI LIỆU THAM KHẢO & NGUỒN ĐỐI CHIẾU

- Prompt Slot 6 do giảng viên cung cấp: React Hook + Lab 01.

- React documentation: Hooks reference; State: A Component's Memory; Synchronizing with Effects; You Might Not Need an Effect; Referencing Values with Refs; Passing Data Deeply with Context.
- Vite official Getting Started – scaffold, dev/build/preview và Node compatibility.
- JetBrains IntelliJ IDEA documentation – React/Vite integration, Node runtime, plugins và Terminal.
- React-Bootstrap documentation – Bootstrap components used in this project.

Cầu nối sang slot sau: Project hiện dùng localStorage/mock data. Khi học Client–Server Communication, data source sẽ chuyển dần sang HTTP request → Spring Boot REST API → JSON response, trong khi phần lớn component architecture, form, state và UI có thể tiếp tục tái sử dụng.